Python for Java programmers

A few minutes tour of the syntax

With examples from the N-queens lab

Python at a glance

	Java	Python
Blocks	{ … }	indentation, after a colon :
End of a statement	;	the end of the line
Comments	// …	# …
Documentation	/** … */	"""…""" (a docstring)
No value	null	None
Booleans	true, false	True, False
Logic	&&, ||, !	and, or, not
The current object	this	self
Running a program	javac, then java	python file.py

Variables, types and strings

JAVA

```java
int n = 8;
double half = n / 2.0;
String name = "queens";
String msg = "N = " + n + ", " + name;
final int MAX = 100;
```

PYTHON

```python
n: int = 8          # or simply: n = 8
half = n / 2        # / always gives a float
name = "queens"     # or 'queens'
msg = f"N = {n}, {name}"
MAX = 100           # a constant, by convention
```

- A variable is created by assigning to it: there are no declarations.
- Type hints such as **n: int** are optional. Python ignores them when it runs; the type checker mypy uses them.
- An **f-string** inserts values into a string: **f"N = {n}"**.

Conditions and loops

JAVA

```java
if (row == 0 && col > 0) {
    count++;
} else if (row == col) {
    count--;
} else {
    count = 0;
}
for (int i = 0; i < n; i++) { ... }
for (Position p : queens) { ... }
while (!queue.isEmpty()) { ... }
```

PYTHON

```python
if row == 0 and col > 0:
    count += 1
elif row == col:
    count -= 1
else:
    count = 0
for i in range(n):      # 0, 1, ..., n - 1
    ...
for p in queens:
    ...
while queue:            # empty means false
    ...
```

- A colon opens a block, and **indentation** (4 spaces) delimits it.
- No **++** or **--**: write **count += 1**.
- **range(n)** counts 0 to n − 1. An empty list, **0** and **None** count as false.

Functions

JAVA

```java
static boolean attacks(int r1, int c1,
                       int r2, int c2) {
    int dr = Math.abs(r1 - r2);
    int dc = Math.abs(c1 - c2);
    return r1 == r2 || c1 == c2 || dr == dc;
}

boolean b = attacks(0, 0, 3, 3);  // true
```

PYTHON

```python
def attacks(r1: int, c1: int,
            r2: int, c2: int) -> bool:
    dr = abs(r1 - r2)
    dc = abs(c1 - c2)
    return r1 == r2 or c1 == c2 or dr == dc


b = attacks(0, 0, 3, 3)          # True
```

- **def name(parameter: type) -> return_type:** defines a function. Functions can live outside classes: no **static** needed.
- Parameters can have default values: **def solve(n: int, algorithm: str = "bfs") -> None**.
- Arguments can be passed by name: **attacks(r1=0, c1=0, r2=3, c2=3)**.

Classes

JAVA

```java
public class MoveState {
    private int[] rows;
    public MoveState(int n) {
        rows = new int[n];
    }
    public void moveForward(int col) {
        rows[col]++;
    }
}
MoveState s = new MoveState(8);
s.moveForward(3);
```

PYTHON

```python
class MoveState:
    def __init__(self, n: int) -> None:
        self.rows: list[int] = [0] * n

    def move_forward(self, col: int) -> None:
        self.rows[col] += 1

s = MoveState(8)        # no "new"
s.move_forward(3)
```

- **__init__** is the constructor, and **self** is Java's **this**, written as the first parameter of every method. Attributes are created by assigning to **self.name**.
- No **public** or **private**: a leading underscore, as in **_items**, marks something internal.

Equality and printing

JAVA

```java
@Override
public boolean equals(Object o) {
    return o instanceof MoveState s
        && Arrays.equals(rows, s.rows);
}
@Override
public String toString() { ... }

a.equals(b)    // same value
a == b         // same object
```

PYTHON

```python
def __eq__(self, other: object) -> bool:
    return (isinstance(other, MoveState)
            and self.rows == other.rows)

def __str__(self) -> str:
    ...

a == b       # calls __eq__: same value
a is b       # same object
print(a)     # calls __str__
```

- Python's **==** compares values, like Java's **equals**; **is** compares identity, like Java's **==**.
- **__str__** plays the role of **toString**: **print** and **str()** call it.
- Objects in sets or used as dictionary keys also need **__hash__**, like **hashCode** in Java.

Lists, tuples and dictionaries

JAVA

```java
List<Integer> rows = new ArrayList<>();
rows.add(3);
int last = rows.get(rows.size() - 1);
List<Integer> copy = new ArrayList<>(rows);

Map<String, Integer> sizes = new HashMap<>();
sizes.put("queens", 8);
boolean has = sizes.containsKey("queens");
```

PYTHON

```python
rows: list[int] = []
rows.append(3)
last = rows[-1]            # from the end
copy = list(rows)          # or rows.copy()

sizes: dict[str, int] = {}
sizes["queens"] = 8
has = "queens" in sizes
```

- A **tuple** is a fixed group of values: **p = (2, 5)**. It cannot be modified, and it can be unpacked: **row, col = p**.
- **len(rows)** is the size, **rows + [4]** is a new list, and **rows[1:3]** is a slice: the elements at indexes 1 and 2.
- **x in collection** tests membership, for lists, tuples, sets and dictionaries.

Interfaces and generics

JAVA

```java
interface Problem<S> {
    S initialState();
    List<S> successors(S state);
    boolean isGoal(S state);
}

class MoveProblem
        implements Problem<MoveState> { ... }

static <S> List<S> bfs(Problem<S> p) { ... }
```

PYTHON

```python
S = TypeVar("S")

class Problem(Protocol[S]):
    def initial_state(self) -> S: ...
    def successors(self, state: S) -> list[S]: ...
    def is_goal(self, state: S) -> bool: ...

class MoveProblem(Problem[MoveState]): ...

def bfs(problem: Problem[S]) -> list[S] | None:
    ...
```

- **TypeVar("S")** declares a type parameter, like **<S>** in Java, and **list[S] | None** means a list of states, or **None**.

- Inheriting from a **Protocol** plays the role of **implements**: the type checker reports any missing method. It is optional: any class with the right methods also fits.

Running your code

JAVA

```java
import java.util.ArrayDeque;

public class Main {
    public static void main(String[] args) {
        int n = Integer.parseInt(args[0]);
        ...
    }
}
```

PYTHON

```python
import sys
from collections import deque
from queens import MoveProblem    # queens.py

if __name__ == "__main__":
    n = int(sys.argv[1])   # argv[0] is the file
    ...
```

- Every file is a **module**: **from queens import MoveProblem** imports a class from queens.py.
- The block under **if __name__ == "__main__":** runs only when the file is run directly, like **main**.
- No compiling: **python main.py 4 place dfs** runs the program, and Ctrl+C stops it. **mypy --strict .** checks the types before you run.

Pitfalls for Java programmers

- **Division**: **7 / 2** is **3.5**. Integer division is **7 // 2**, which gives **3**.
- **Indentation is syntax**: a line indented differently changes the meaning of the program, or is an error.
- **Forgetting self**: every method takes **self** first, and attributes are always written **self.name**.
- **Aliasing**: **b = a** makes both names refer to the same list, as in Java. **list(a)** or **a.copy()** makes a new one: that is why the lab's states have a **copy** method.
- **Mutable default values**: never write **queens: list[Position] = []** as a default, because every call shares that one list. Use **None** and create the list inside, as **PlacementState** does.